Project 3: Inheritance, O, L, I Principles

About

This programming project lets you practice writing objects with inheritance in python, and apply the L, I and O principles, as well as using the pygame library.

Learning Objectives

1. Using Inheritance
2. L, I, and O Principles
3. Practice using Pygame

Estimated Size

4 Python files:

- First file is: **`my_sprite.py`**
- Second file is: **`colliding_object.py`**
- Third file is: **`moving_vehicle.py`**
- Fourth file is: **`main.py`**
- Some functions require you to call other functions you implemented in the same file.

Extra Restrictions?

1. The **maximum** grade you may receive from the autograder will be **60/100.** The remaining **40 points** will come from **handgraded** evaluation of your **Inheritance** and following **design principles,** as well as the content of your **main.py**
    - The rubric we will use in Gradescope will be posted later for your reference.
2. Any lines of code that execute your functions (like testing code) needs to be put in **the main method branch for python:**
   `if __name__ == '__main__':`
3. You may need to familiarize yourself with the pygame documentation to complete the assignment: https://www.pygame.org/docs/
4. To make writing the autograder easier, please use the exact same attribute and functions names. You can create additional attributes and methods as you see fit, but the ones in the ADTs should match exactly.
5. Please use the following lines whenever you **`import pygame`** in a file to stop the prompt pygame displays from potentially crashing the autograder

```
import os
os.environ['PYGAME_HIDE_SUPPORT_PROMPT'] = "hide"
import pygame
```

Table Of Contents

Project 3: Inheritance, L, I, O Principles
- About
- Learning Objectives
- Estimated Size
- Extra Restrictions?

Table Of Contents

Part 1 - my_sprite ADT
- Complete my_sprite.py (20 points)

Part 2 - colliding_object ADT
- Complete colliding_object.py (20 points)

Part 3 - moving_vehicle ADT
- Complete moving_vehicle.py (20 points)

Part 4 - Main File & Hand Graded Portion
- Complete main.py (10 points)
- Follow SOLID Design Principles (30 points)

Additional Info about Game Programming
- Game Development Software
  - Languages
- Classes I recommend

Submitting to Gradescope
- Auto-Grader
- Unlimited attempts
- Academic Honesty

Part 1 - my_sprite ADT

Complete my_sprite.py (20 points)

Here is an ADT of the my_sprite class, follow the ADT in your implementation of the class. make sure the class is named **my_sprite** and is in a file called **my_sprite.py**

Remember that this is an ADT, it is the bare minimum of what a class must have to "qualify" as an object of that type. You may need/want to create additional methods during the implementation process.

To make writing the autograder easier, **please use the exact same attribute and functions names**. You can create additional attributes and methods as you see fit, but

the ones in the ADTs should match exactly. Normally when implementing an ADT this is not required, it's only required in this project because of the autograder.

- `__init__ (image_fname:str, loc:tuple[int, int]=(0,0))`
  - creates an object of type `my_sprite`.
    - `image_fname` stores the name of an image file that will be used to represent the sprite on the screen.
    - `loc` is a tuple that indicates the location of the object.
  - Uses `pygame.Surface.get_width()` to obtain the width of the object
  - Uses `pygame.Surface.get_height()` to obtain the width of the object
- `get_image() → pygame.Surface`
  - returns an image that can be blitted to represent the object on a pygame canvas
- `get_width()→ int`
  - returns the width of the object
- `get_height() → int`
  - returns the height of the object

Part 2 - colliding_object ADT

Complete colliding_object.py (20 points)

Here is an ADT of the colliding_object class, follow the ADT in your implementation of the class. make sure the class is named **colliding_object** and is in a file called **colliding_object.py**

Remember that this is an ADT, it is the bare minimum of what a class must have to "qualify" as an object of that type. You may need/want to create additional methods during the implementation process.

- `Inherits from **my_sprite**. a my_sprite with collision detection.`
- `__init__ (image_fname:str, loc:tuple[int,int] = (0,0))`
  - Exactly like the constructor for my_sprite, but has a **`bounding_box`** attribute as well
  - Uses `pygame.Rect(locx,locy,w,h)` to create a bounding box around the object, at position [locx, locy], with width=w and height=h
- `get_bounding_box() → pygame.Rect`
  - returns the bounding box that is bounding the object.
- `is_colliding_with(co:'colliding_object') → bool`

- uses `pygame.Rect.colliderect(pygame.Rect)` to determine if this object is colliding with some other colliding_object. Returns True if the objects are colliding, and returns False otherwise

---

Part 3 - moving_vehicle ADT

Complete moving_vehicle.py (20 points)

Here is an ADT of the moving_vehicle class, follow the ADT in your implementation of the class. make sure the class is named **moving_vehicle** and is in a file called **moving_vehicle.py**

Remember that this is an ADT, it is the bare minimum of what a class must have to "qualify" as an object of that type. You may need/want to create additional methods during the implementation process.

- `Inherits from` **`colliding_object`**`. A colliding_object that can change it's location.`
- `__init__ (image_fname:str, loc:tuple[int, int]=(0,0))`
  - Exactly like the constructor for colliding_object
- `set_location(loc:tuple[int, int])`
  - changes the position of the object to loc
  - changes the position of the object's bounding box to loc

Part 4 - Main File & Hand Graded Portion

Complete main.py (10 points)

Write a file called `main.py` similar to what you used in Lab 2 and Lab 3. At **minimum** your main file should:

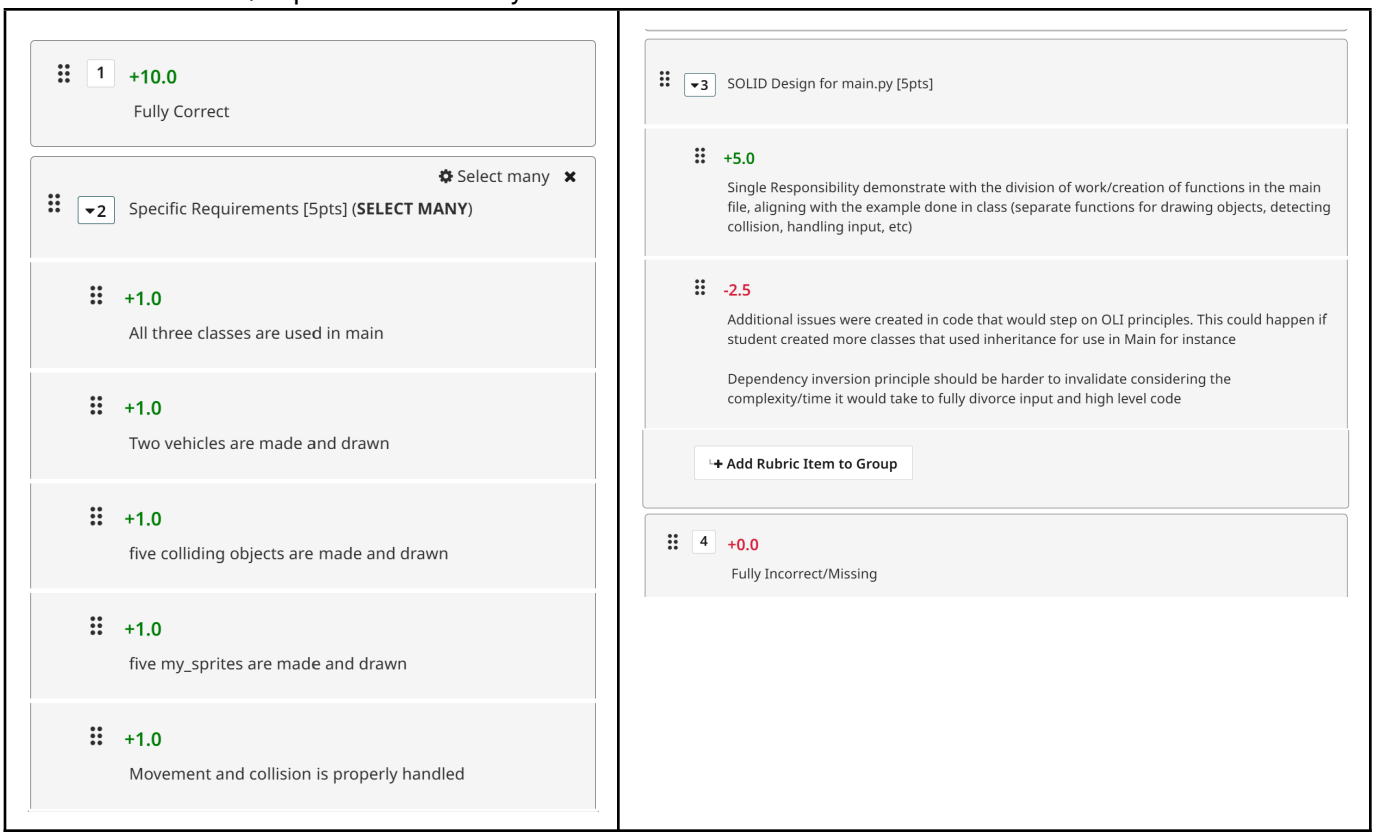
- [ ] **have a function called main() that starts execution of your code**
- [ ] Use all three of the classes created in the prior parts
- [ ] Have two controllable vehicles.
- [ ] Have five colliding objects.
- [ ] Have five my_sprites used for decoration.
- [ ] Have properly handled movement and colliding functionality.
- [ ] Follows the SOLID design principles talked about in class, specifically how those ideas can relate to the functions of your **main.py** file

Here is a collection of simple sprites we are used to working with in the labs: images_orig.zip. I highly encourage you to go and find your own sprites however. Don't feel restricted by the use of

the term "vehicle" in the class, all of our code just deals with moving objects, objects with collision, and decorative objects, your program does not have to look or "play" in any specific way besides having those three types.

- **IF YOU REPLACE THE IMAGES OR ADD ADDITIONAL ONES**: make sure you upload the images folder along with your code, and that your code uses the proper **relative paths** (more info idk why it's from an earth science lab website it just had the best explanation) so that the graders can download your program and run it straight away

Rubric for main.py section. This rubric can change slightly later on, but most of the point distribution/requirements will stay the same



Follow SOLID Design Principles (30 points)

You code should follow all five of the SOLID design principles, with a focus on Single Responsibility, the Liskov Substitution principle, and the Inheritance Segregation principle.

You can use this rubric as reference for what we will be specifically looking for in grading. Note that the rubric might not be final, and we could change the wording or specifics of the rubric as we grade. The general requirements and overall point distribution will remain the same though.

Q3.1 Inheritance used

5 points

1 **+5.0**

Correct use of Inheritance on both objects (my_sprite->colliding->moving)

2 **-20.0**

Code passes test but **did not use inheritance to solve the problem**

+ Add Rubric Item | Create Group | Import...

Q3.2 Doesn't needlessly redefine methods

5 points

1 **+2.5**

get_image, get_width, and get_height are only declared in My_sprite.

get_bounding_box and is_colliding_with are only declared in Colliding_object

2 **+2.5**

attributes are created by calling the constructor of the base class

3 **-10.0**

get_image, get_width, get_height, get_bounding_box, or is_colliding_with **are redefined in the inheriting object**

-OR-

super.__init__() is **not** called, or some attributes already present in My_sprite are unnecessarily created in Colliding_object

Q3.3 S & D

10 points

1 **+10.0**

Fully Correct, solution maintains the single responsibility principle for all classes created (not just the required classes, but also any additional classes)

Dependency Inversion should also be maintained to the best extent, although this requirement is more forgiving due to the complexities in making code that fully follows this principle

This applies to all classes **except** main.py which is already being evaluated in a different section of the rubric

2 **+5.0**

Solution mostly maintains the principles, but makes some mistakes or choices that lead to a less ideal system. Alternatively, solution keeps in mind S but not D, or D but not S.

3 **+0.0**

Solution voids S or D principle in a way that clearly hinders the design/use of the classes.

I imagine this will be uncommon if student followed the instructions/layout for the project

Q3.4 Liskov Principle

10 points

1 **+10.0**

Correct, Objects of derived classes can do just as much (if not more) than their parent class.

Should be easy to get considering the layout of the instructions.

2 **+0.0**

Solution voids Liskov principle in a way that clearly hinders the design/use of the classes. Objects of the derived class clearly can do less than objects of the parent class.

I imagine this will be uncommon if student followed the instructions/layout for the project

Additional Info about Game Programming

Pygame is a tool of convenience for this class, offering us practice with object oriented programming in a very interactable and in immediately realized space. We know python, and we understanding what a game should do and look like, so we can use that learn OoP.

For those of you interested in continuing their learning on game programming, I always say the best language/system to use is whatever one you will actually use. The best practice you can get is to commit time to working on something you are passionate about. That being said python is a great language for learning and quick scripting, but it is not considered the best language if you want to make or publish a game seriously. It's not that you can't but you are fighting a bit of an uphill battle.

Game Development Software

Personally I've been having fun developing in Godot: https://godotengine.org/, it has some limitations for the less programmatically inclined, but that shouldn't be too much of a problem if you are already taking computer science courses.

I have also used unity a fair bit: https://unity.com/, but it's not without it's controversy especially with their asset store and licensing changes. One of it's biggest advantages is it's low bar of entry and the prominence of their userbase in making tutorials and high quality assets/resources still makes it desirable to some extent.

Unreal engine is another option: https://www.unrealengine.com/en-US, this is the one that I have most experience using. Unreal is pretty resource greedy, and hardware requirements can quickly balloon, along with being fairly complex in it's workflow. It can allow for some very high quality products though, if you can put in the time, which is I ended up using it most often in my experiences.

Languages

Each of those tools have their own quirks, scripting, and tools you would need to learn during development, as a start each of them has a programming language you would benefit from learning. Godot uses GDScript, C#, and C++. Unity uses C# (c-sharp), Unreal uses C++ (c-plus-plus), and their blueprint system. From someone coming from Java or Python, a C based language should not be a large leap to learn, mainly just learning some unique quirks and the different syntax, but fundamentally they will work very similarly.

If you need a starting place project for learning a language, I usually recommend trying to program a calculator to start. As it can be done in the bare minimum super super simply, but there are so many complexities/added features you can add onto it but still quickly evaluate if it is working or not.

Classes I recommend

COMPSCI 576: Game Programming. Usually offered once an academic year, general information class open to undergraduates at the higher level (I took it when I was a senior). I have been asking to teach it every time the faculty get asked about their teaching preferences, we will see if I ever do

COMPSCI 325: Introduction to Human Computer Interaction. Most of the class focuses on design and user usage, especially menu and website design, but the design rules apply for menus and user interfaces in games.

COMPSCI 373: Introduction to Computer Graphics. Usually taught in the spring. Computer graphics is a huge part of game programming, and will be super useful when you need to design your own tools and systems on top of an existing game development software suite.

There are also some classes that might seem relevant but don't really have much to do with games as we talk about them. AI, Machine Learning, Simulation classes, while they can be relevant are currently not commonly used in games (save the occasional like Who's Lila) the term AI used in game design doesn't directly relate to the field of study AI in computer science.

Hampshire college has a whole game design program, which you can find more information on here: https://www.hampshire.edu/academics/areas-of-study/game-design-and-development
As a part of the five college program, you may take classes at Hampshire College (or Holyoke, Smith, or Amherst) for no additional tuition fee. This is phenomenal to me, but very few UMass students take advantage of it, mostly it's used by students from the other colleges taking classes at UMass (which makes sense considering our size and how many different classes we offer)

To learn more about these system here: https://www.fivecolleges.edu/

Submitting to Gradescope

Submit all your files to gradescope at once. Wait for the autograder and review any failed tests. Make the appropriate changes in your code and resubmit as many times as you need before the deadline. While working on each question, you can submit that file alone and receive feedback. In the end you must submit all files at once to receive full credit. Your highest scoring submission will be recorded as your grade. Note that each failed test includes text at the bottom giving hints as to why that test failed.

Auto-Grader

To submit all files at once, you can:

- Select all files you want to submit, then drag and drop them in the submission area when prompted.
- Browse your files when prompted by Gradescope and select all files you want to submit.
- Alternatively, compress all the files you want to submit into a single zip file (the zip file name does not matter); then submit that zip file via drag-and-drop or browsing.
- You can also compress the folder that contains all the files you want to submit into a single zip file (the zip file name does not matter), and submit that zip file via drag-and-drop or browsing.

Please note that for the programming assignments, **every student must submit individually**. If you truly worked on your program by yourself, your code will not be the same with other students'. Autograder checks for unique similarities between two students that are not shared by other students: if the submissions of most students are similar, that means they are using common knowledge such as code from lecture slides or textbook, which is NOT considered plagiarism. However, if there is unique and strong similarity (e.g. more than 90%) between two students that are not shared by other students, that will be flagged for further investigation of **academic dishonesty**. Students working on code by themselves have yet to be falsely identified as overly similar, so you don't need to worry about this as long as each line of code is written by yourself.

Unlimited attempts

You have unlimited attempts for our auto-graded assignments, until the due date passes. This means that you should submit early and often in order to be confident that the code you write will pass all of the autograder tests.

Academic Honesty

All work that is completed in this assignment is your own. You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way. What you submit **must be your own work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.